1. Supervised Learning

In supervised learning, the AI is given many examples of the types of questions it is expected to answer, and the correct answer for each question. By analyzing these, it attempts to discover the patterns that match questions to answers. The answer is called the Label. Since supervised learning requires the correct answer, it requires labeled training data, or data for which the right answer is known. In the example below, the data shows an AI whether a person is an Adult or a Child based on the *Number of Countries* the person has visited, the *Years in School*, and the *Height* in feet. The label is *Who am I? - Adult* or *Child*.

Concept
Label

The correct answer for a question, provided as a part of the data. An AI uses this information to train itself.

Samples

Each row is an example for the AI to Train with

Number of Countries	Years in School	Height	Who am I ? (Label)
20	15	5.2	Adult
2	3	3.5	Child
10	12	4.9	Adult

Context

Example of Supervised Learning

If a bank wants to build an AI to predict which customer is likely to leave the bank, it can use historical data about each past customer, with a label that states whether that customer left the bank. With such a dataset, the AI can learn the difference between customers who are likely to leave and customers who are likely to stay. Once the AI is trained, the resulting Model can be used to predict whether a new customer will leave or stay.

Figure 2.6 Bank statement of past transactions

2. Unsupervised Learning

In this form of learning, the AI is not given the right answers but is expected to learn patterns within the data rather than match a data pattern to an answer. A good example of unsupervised learning is detecting anomalies or unusual behavior.

If a credit card is stolen, it is likely that the thief will use the credit card in a way unlike what the owner did. An AI that has historical information about the purchase patterns on this credit card can establish a baseline - what normal purchasing behavior for this customer looks like. Once the thief starts buying things with the credit card, the AI can detect these purchases as unusual and flag an alert.

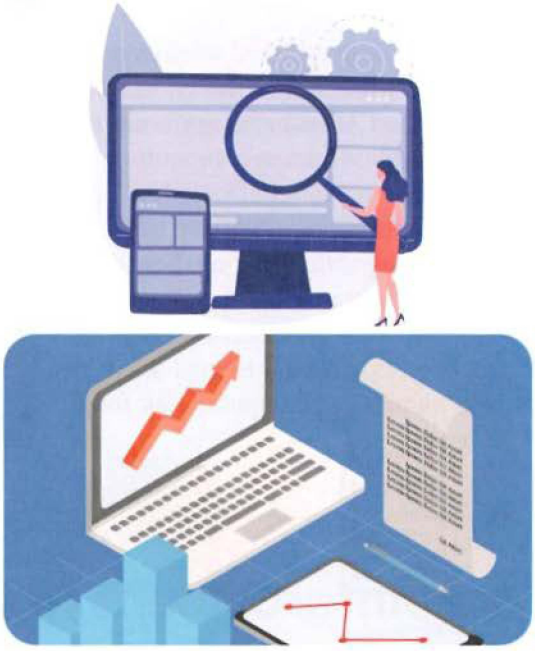


Figure 2.7 Examples of unsupervised learning

Another popular class of algorithms in unsupervised learning is clustering. Given a dataset, clustering refers to dividing it into groups that are similar. One such metric of similarity is distance. Using this metric, the algorithm will cluster points that are close to each other. Figure 2.8 shows an example where the algorithms grouped the points into 3 clusters, where each cluster is represented by a different color.

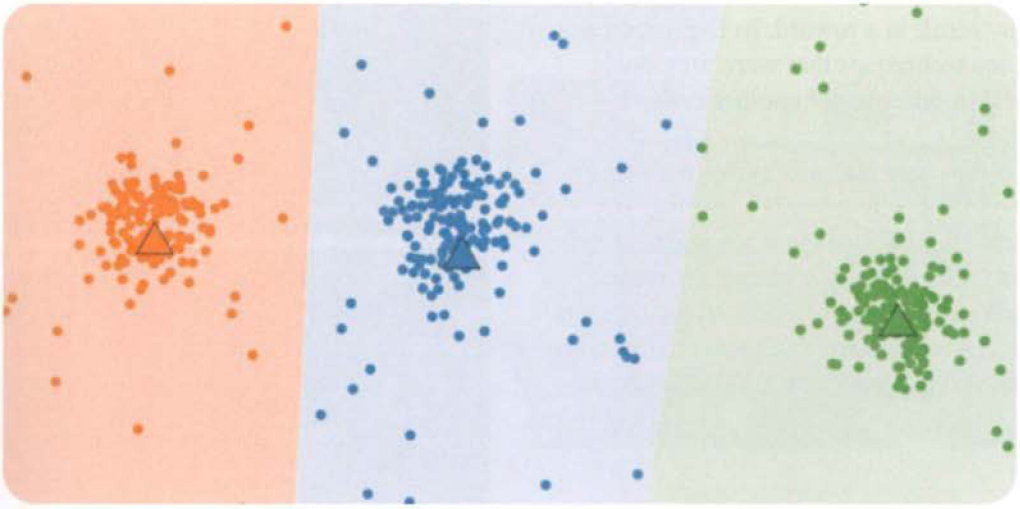


Figure 2.8 Example of cluster grouping

Context

Alpha GO

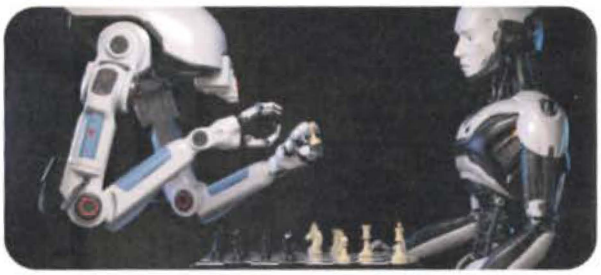
AlphaGo is an artificial intelligence (AI) agent that is specialized to play Go, a Chinese strategy board game, against human competitors. AlphaGo is a Google DeepMind project. The ability to create a learning algorithm that can beat a human player at strategic games is a measure of AI development. This AI has been extensively trained using both human and computer play for it to learn.

Figure 2.9 The Chinese strategy board game Go

3. Reinforcement Learning

In Reinforcement Learning, the AI experiments and is given a reward for success. Over time, the AI remembers what experimental steps led to success and what did not, and becomes better at optimizing for the reward. Reinforcement learning AIs operate in two modes - Exploration and Exploitation. In Exploration, the AI tries new techniques to see what may result in a reward. In Exploitation, the AI uses techniques that were previously successful in order to get another reward.

Most AIs that play and win games are using some form of Reinforcement Learning. In this case, techniques are new strategies in the game, and the reward is winning the game. By playing a very large number of games over time, the AI can accumulate a series of winning strategies and become very good at the game.



Figures 2.10 Examples of reinforcement Learning

What is Neural Network?

Neuron

Before we can understand what a Neural Network is, let's explore a single Neuron. Figure 10.1 shows an example of a basic neuron, called a Perceptron. It takes in several values, applies some kind of mathematical function to them, and generates an output. Each input has a weight applied to it.

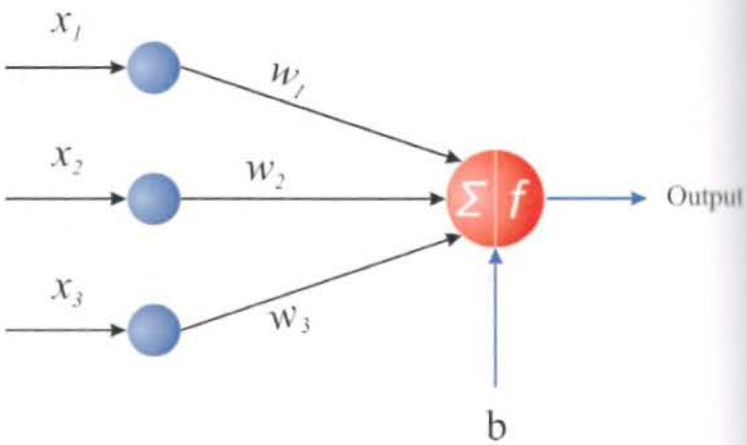


Figure 10.1 An example of a single neuron

By changing the weights, the neuron can decide to prioritize (or take more notice of) some inputs over others. It can even ignore some inputs entirely by setting their weights to zero. Figure 10.2 shows an example of what weights and a function can look like.

The process of training a neural network involves changing these weights and finding the best value for the weights. While this may seem simple for a single neuron, real world neural networks have millions of neurons and many millions or even billions of weights, making training challenging.

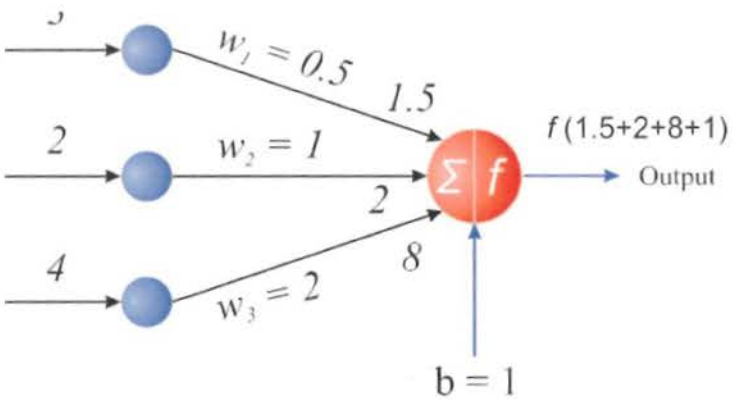


Figure 10.2 Example weights and function for a neuron

A Neural Network

A neural network is, as expected, a network of neurons. Figure 10.3 shows a simple example of a neural network, called a Multi-Layer Perception (MLP). An MLP is also called a fully connected network. The network has an *Input Layer*, an *Output Layer*, and some number of *Hidden Layers* (in our example there is one hidden layer).

Each layer can have many neurons. In the example, the input layer has 3 neurons, the hidden layer has 4 neurons, and the output layer has 2 neurons. Each neuron takes inputs from the layer before it and sends its output to the next layer. In an MLP, each neuron is connected to every neuron in its neighboring layers.

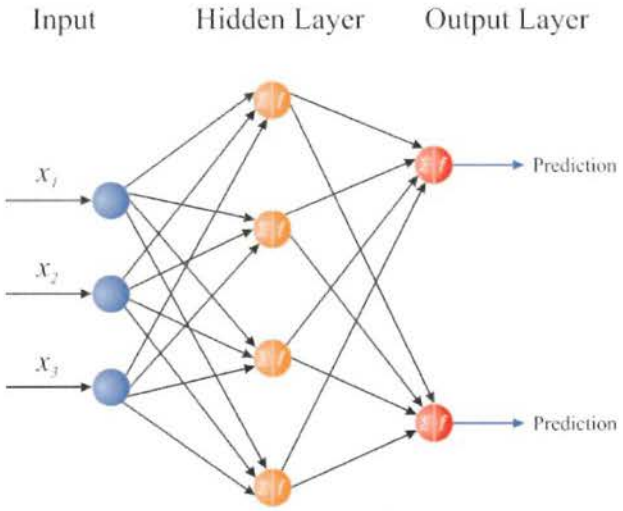


Figure 10.3 An example network of neurons

Size of Real World Neural Networks

The number of neurons in a neural network depends on its architecture. The number of hidden layers, the types of layers, the number of neurons in each layer etc all define a neural network's architecture and determine its size. In the real world, neural networks are used in varied applications such as image classification, language translation, question and answering, and more. A typical image classification neural network in the real world has 5,000 to 60,000 neurons. The architecture of neural networks differs between different applications, and each one is quite sophisticated in itself. The complexity of training a neural network is measured by the number of parameters rather than the number of neurons. The number of parameters is the number of weights in the network. Real world neural networks can have millions or even billions of weights and can take hours to days (or sometimes even weeks) to train.

What is Deep Learning?

There are many types of neural networks, and they differ in various ways. Some ways that neural networks differ include the structure of the neurons themselves, the way neurons are connected, and of course the number of layers. This is where deep learning comes in. Neural networks with a very large number of layers are called *Deep Neural Networks*, and the technology to train and manage deep neural networks is called *Deep Learning*.

There is a popular *AI Challenge* in the images domain called *ImageNet*. Researchers all over the world compete and submit their models to it every year. The dataset consists of more than 14 million images. Deep neural networks have been winning this competition since 2010. Every year, the depth of the network that wins the challenge keeps increasing.

Figure 10.4 shows the history of neural networks over the past few years. You can see that the depth of these networks has increased substantially as researchers have discovered new ways to construct networks that can train in a stable and reliable way while having lots of layers. The larger the network, the more likely that it can tackle a tough problem with rich data like images and video.

Other factors have also helped researchers build deeper neural networks. Datasets are getting bigger. As seen in previous chapters, data can now be gathered from everything from sensors to cameras. Computers have also gotten more powerful. All of these factors have made it possible to create deeper and more effective neural networks.

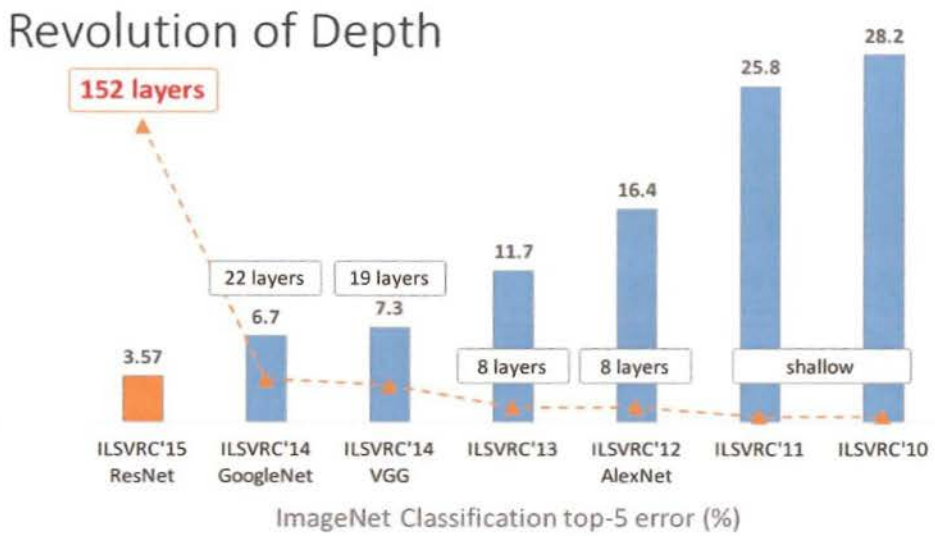


Figure 10.4 The depth of neural networks winning the ImageNet challenge, year by year.

Types of Deep Learning

As mentioned before, there are many types of neural networks. Similarly, there are many types of deep learning that are used to train different types of neural networks. Some common types of deep learning are:

1. *Convolutional Neural Networks* (CNNs): These are popular for processing and classifying images.

2. *Recurrent Neural Networks* (RNNs): These neural networks are good for data that has some type of sequence. An example is a sentence where the order of the words is important to the meaning. A particular type of RNN is a *Long Short-Term Memory network* (LSTM).

Researchers are always experimenting with new neural network architectures and finding the best ones that work. For example, a very recent type of neural network, called a *Transformer*, has been shown to be really good at language problems (like question and answer or summarizing text). This type of neural network can also make some really cool chatbots.

Deep Learning in Self Driving Cars

Self-driving cars use a lot of deep learning technology. A self driving car has many cameras within it that are used to take pictures of the surrounding environment. Using the AIs, the car needs to determine what types of objects are around it and how fast and in what direction the objects are moving. It needs to be able to identify other cars, pedestrians, bicyclists, traffic signs, puddles on the road, fires, and anything else it should expect on the road.

The technology used to do this is called object detection and object recognition. The first step is to identify the many objects in the single image. The second step is to identify what each object is. This also needs to be done in real time and very fast. There is no benefit in identifying a pedestrian after the car has run into them!

To accomplish this feat, self-driving cars use deep learning AIs that have been trained on massive numbers of images, trying to capture everything that the car may encounter on the road. When the car is driving, it uses these models to determine the objects around it.

Figure 10.5 Self driving cars and object detection

Supervised or Unsupervised?

Deep learning gets its name because it is based on deep neural networks. Many of the deep learning techniques and uses are supervised learning. In the rest of this chapter, we will focus heavily on image classification with deep learning, which is a supervised learning problem.

However, deep learning is not limited to supervised learning. It can also be used for unsupervised learning problems.

Image Classification

A very common use of deep learning is for image classification. Figure 10.6 shows how this works. The concepts are identical to what was covered in Chapter 5 about classification. The only difference is that, instead of feeding the AI a table of data to learn from, we feed it a set of images. Since this is a supervised learning AI, there are still labels. Each image is now associated with its label (the right answer).

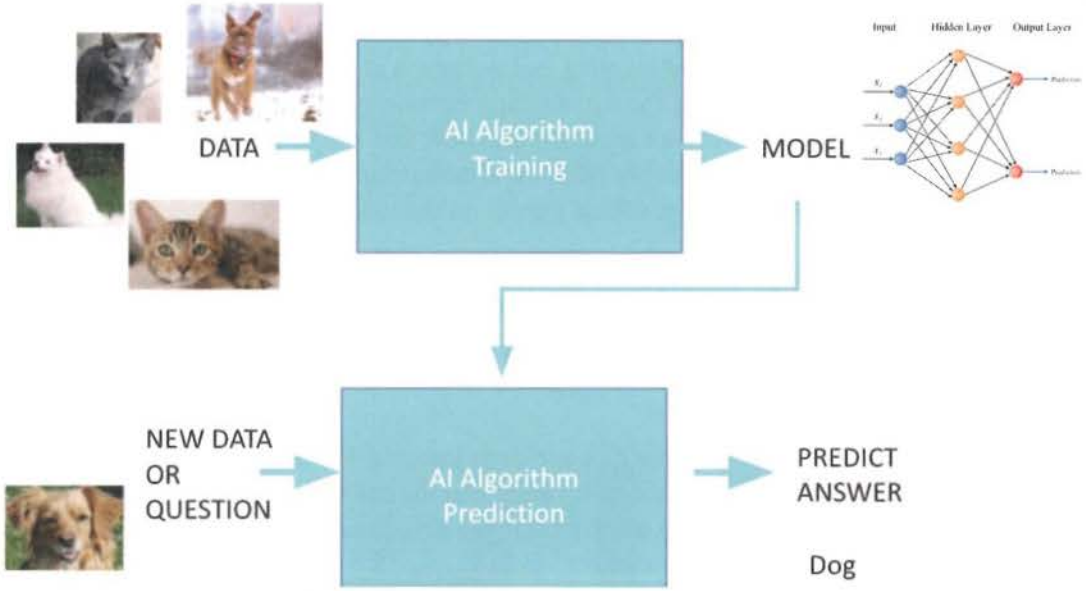


Figure 10.6 Image Classification

Since image classification, even though it uses deep learning, is still classification, everything covered so far about classification still applies. *Accuracy* and *Confusion Matrix* can still be used to measure a deep learning image classification AI. Training and prediction mean exactly what they did before. The model in this case is the trained neural network with the values of all of its weights. The feature in this case is the image or a subset of it, depending on what type of deep learning is used. The dataset is the set of images, and the accuracy is still computed as before, using a random subset of the images as a test dataset.

How does a Neural Network Process Images?

Before we can learn how neural networks process images, we will need to learn how computers in general process images. Images are stored in computers as a set of pixels, where each pixel value reflects the intensity of color in a small part of the image. Figure 10.7 shows a pixelated image - an image with so few pixels that the human eye can see each pixel. While images like this can be found, for example, in very old video games, most images today contain millions of pixels and the human eye cannot see the pixels (this is also why the images are such high quality).

What is a pixel?

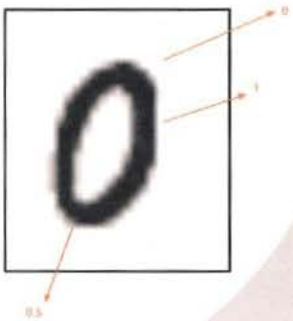
In a black and white image - a pixel is a number that shows the grayness of that part of the image. In Figure 10.8, you can see that the white pixels are 0, the black ones are 1 and the grey ones are 0.5. Sometimes pixel values can also range from 0 to 255.

A color image has more information. Each pixel is actually three values, the *Redness*, *Greenness* and *Blueness* (RGB) of the pixel. These can again range from 0 to a max value like 255.

When a computer processes images, it sees the image as a grid of numbers if the image is black and white, and as three grids of numbers if the image is color.

Have you ever seen a pixelated image?

Figure 10.7 A pixelated image



Each pixel is represented by a number?

Figure 10.8 Gray scale values of pixels in a black and white image

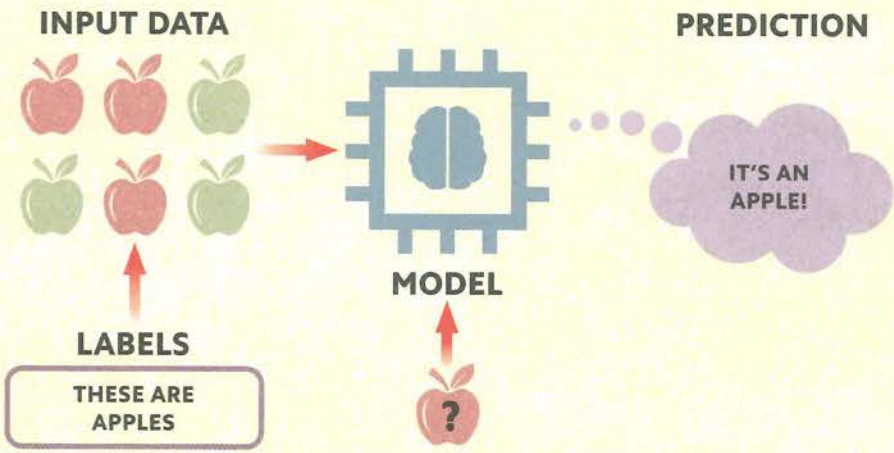
Concept

Pixel

Smallest units of an image are called pixels

MACHINE LEARNING WITH "LABELLED" DATA

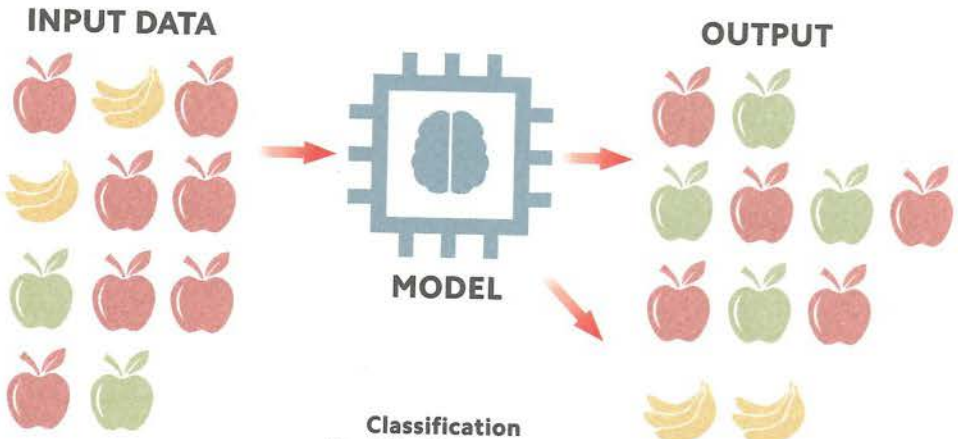
Supervised learning is a type of machine learning in which an AI is trained using a "labelled" training data set (see p.61). Input and output data is labelled by a human so that the AI can learn the relationship between them. The inputs, outputs, and the rule that relates them are collectively known as a "function". During training, weights (see p.78) are adjusted to make the function fit the training data. The resulting function can be used to predict outputs based on new inputs. Supervised learning can be used for classification (see p.66) and regression (see p.67).



Labelled data set
The AI is trained using labelled data – for example, that the inputs are apples.

Test data
After training, the AI is tested on unlabelled input data to check its performance.

Prediction
On the basis of its training data, the AI predicts the label of the input is an apple.



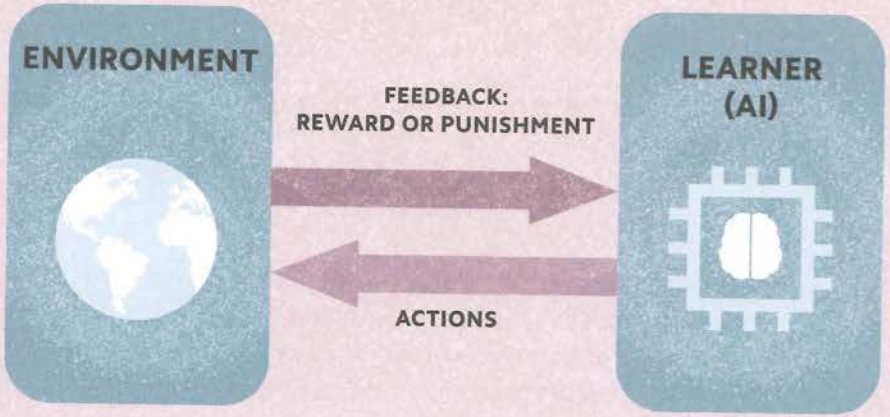
Classification
The AI is able to classify inputs – for example, different fruits – from a large, unlabelled data set.

MACHINE LEARNING WITH "RAW" DATA

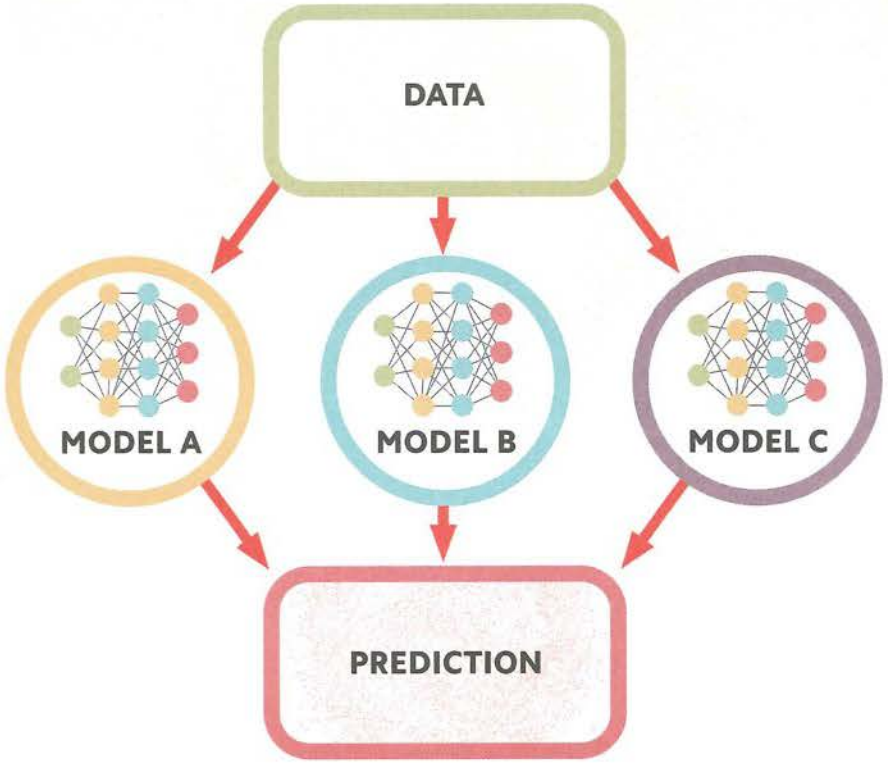
Unsupervised learning is used to discover hidden structures in raw, unlabelled data sets. Although AIs do not understand the relevance of these structures, they may still have real-world meaning. This approach is useful in the early stages of data mining (see p.60), to find patterns in large, unlabelled data sets, which can then be subject to human interpretation. An in-between method, semi-supervised learning, uses partly labelled data sets, which gives better results than entirely unsupervised learning.

LEARNING FROM FEEDBACK

Reinforcement learning is an approach to machine learning (see pp.58–59) in which an AI is taught to perform a task through trial and error. To achieve this, the AI is programmed to recognize "rewards" and "punishments", meaning positive or negative feedback, depending on whether it succeeds or fails. The AI learns that succeeding is good and failing is bad, and repeatedly attempts the task until it is rewarded. For example, an autonomous vehicle trained in this way (see p.122) will be punished – receive negative feedback – until it learns not to go through a red traffic light.

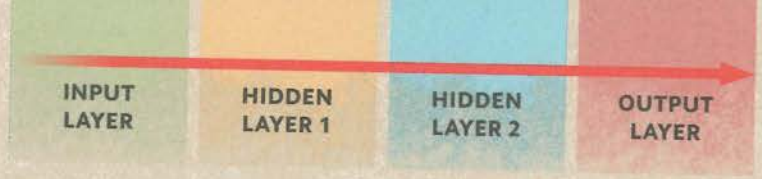


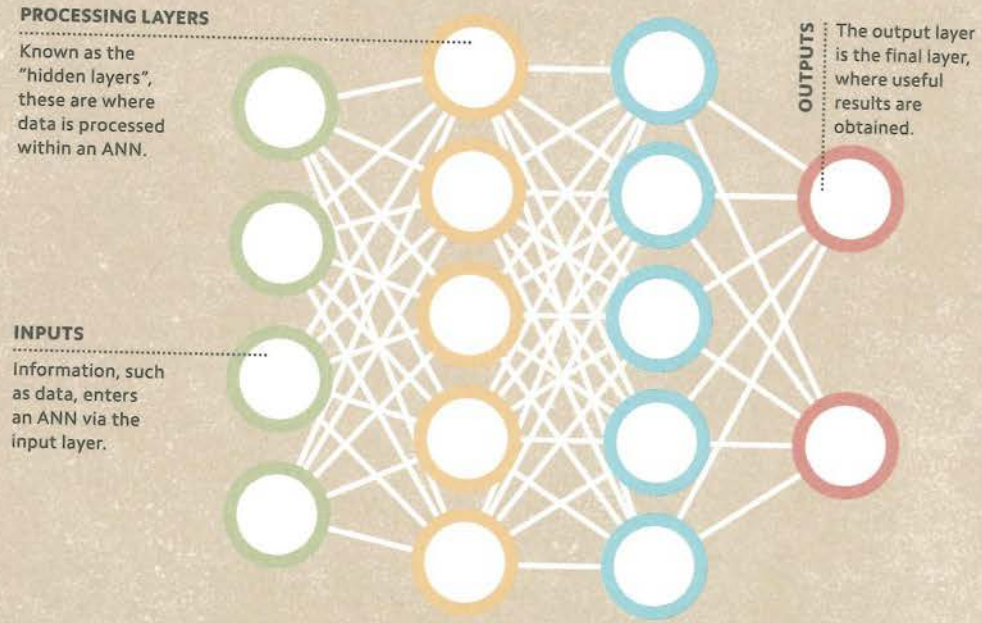
Trial and error
The AI learns to succeed in a task through the consequences of its actions. It will seek rewards and avoid punishment until the task is completed.



WORKING TOGETHER

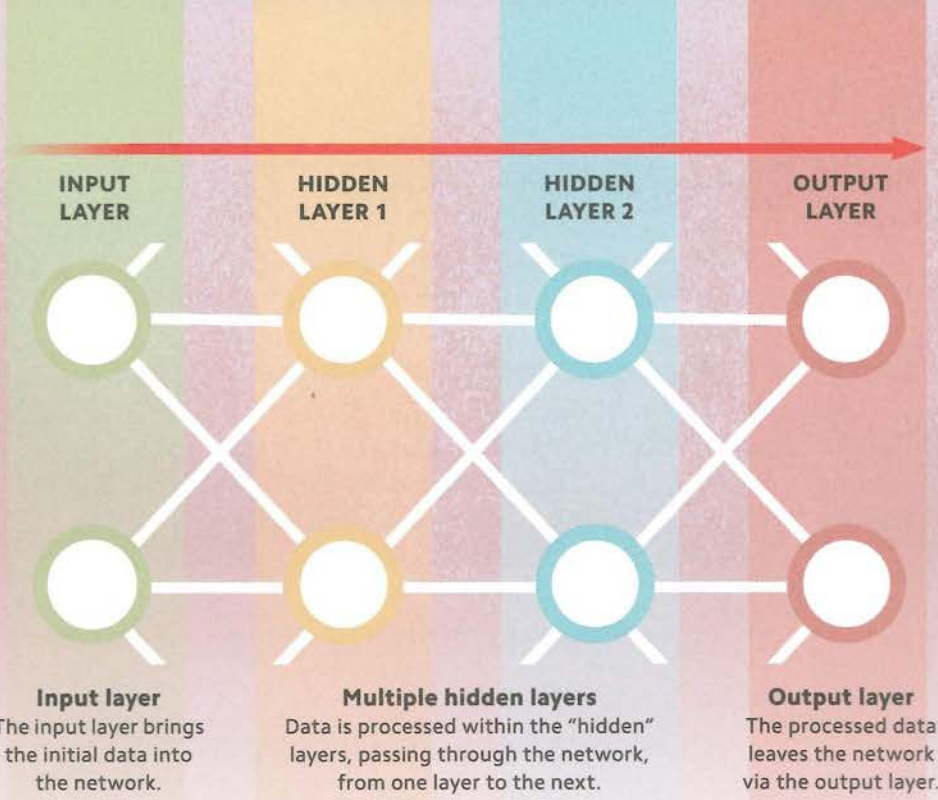
Ensemble learning is based on the idea that combining the outputs of multiple machine-learning algorithms produces a better result than a single model can. Using two or more models that have been built and trained in different ways, for example with different data sets, can "cancel out" their individual weaknesses and generate more accurate predictions. Ensemble learning can be used to "teach" a particular model to improve its predictive performance, but also to assess a model's reliability and prevent a poor one being selected.





THE AI BRAIN

Artificial neural networks (ANNs) are machine-learning models based on algorithms (see p.14). Their structure is similar to that of the brain, consisting of interconnected nodes – artificial neurons – that are organized into multiple "layers". The nodes within each layer receive, process, and send data to the next layer in the network, until an output, or result, is produced. Each node works like an individual microprocessor that can be reprogrammed to handle the data in a desired way. Using training data (see p.61), programmers can teach an ANN to "learn" how to give the expected results, or outcomes.



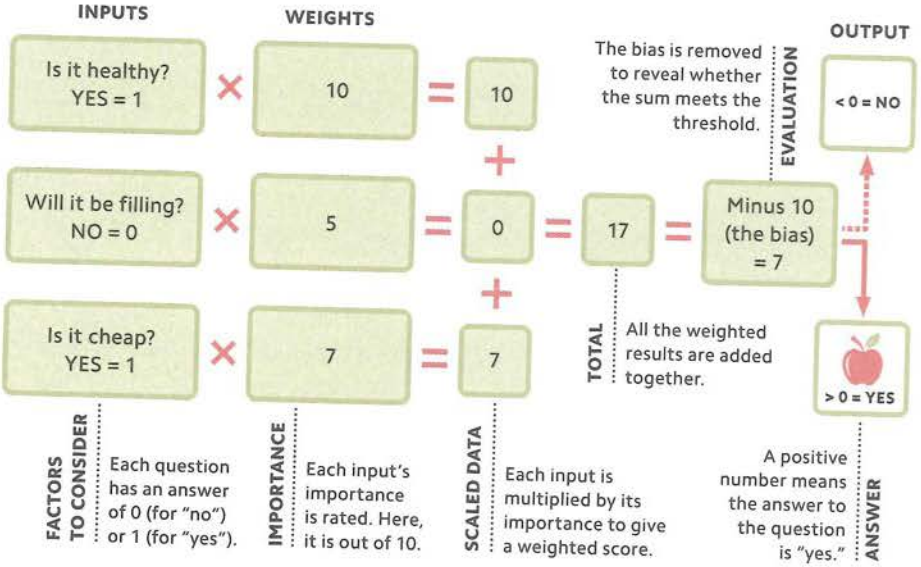
NETWORK STRUCTURE

Artificial neural networks (ANNs) are structured in "layers"– collections of processing nodes that operate together. Data flows from the nodes in one layer to those in the next. The first layer always contains the "input," or incoming data. Next is at least one "hidden" layer, in which the processing takes place. These layers are hidden in the sense that their data is not visible to a user in the way that the network's inputs and outputs are. Finally, the resulting data arrives at the "output" layer. All ANNs share this basic structure, but some are more complex: recurrent neural networks (see p.85) generate connections between nodes in the next or in previous layers, while deep neural networks (see p.86) can have hundreds of hidden layers.

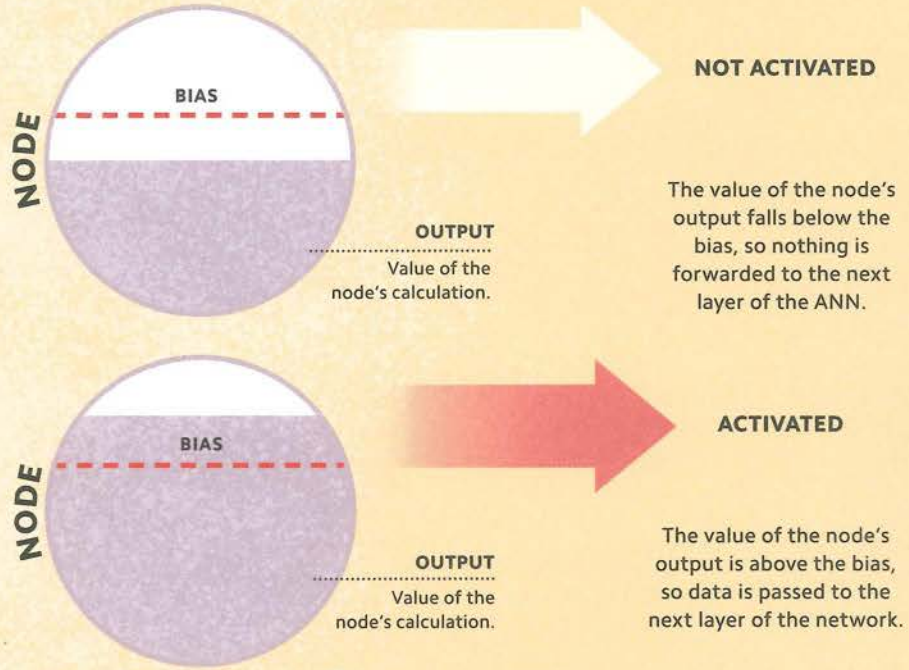
ASSIGNING IMPORTANCE

AI algorithms include variables – mathematical values that can change – that determine how data is processed within an artificial neural network (ANN, see pp.76). When designing and training an ANN, programmers can give these variables greater or lesser influence within the algorithm. This influence is known as "weight". The more weight an input has, the greater its influence over the output. The "bias" (see opposite) determines the threshold at which variables become significant. Adjusting the weights and bias allows the ANN to be fine-tuned to give more accurate results.

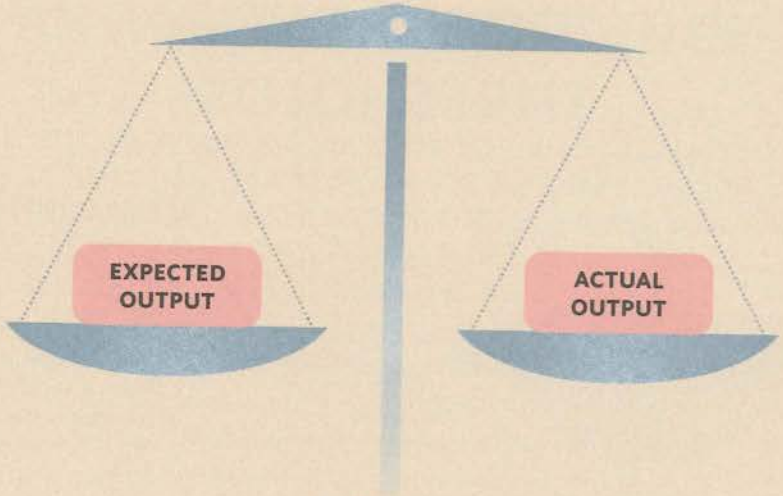
SHOULD I SNACK ON AN APPLE?



GOALS AND THRESHOLDS



An artificial neural network (ANN, see p.76) is made up of layers of "nodes", which receive and process data. Before a node can pass information on to the next layer of nodes, its output data must reach a certain value. This value – essentially a numerical score set by the ANN designer – is known as the "bias". The node can only "activate" and pass on its output data once the bias has been met. If the node is not activated, that path of data transmission stops. Different biases can also be set to direct data to specific nodes on the next layer of the ANN.



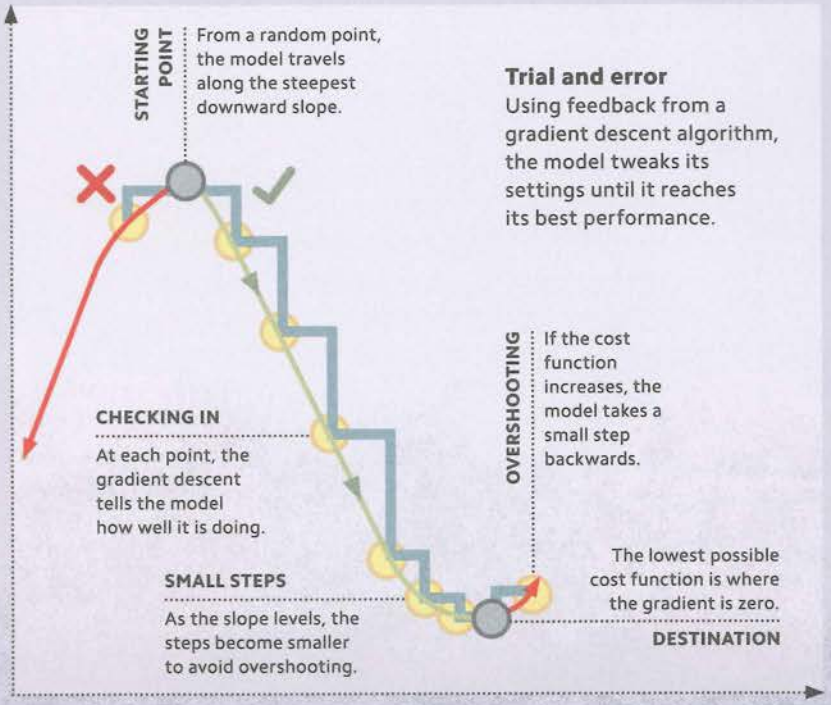
Cost function
The difference between the expected and actual outputs gives the model's performance. The goal is for them to be equal.

MEASURING SUCCESS

The performance of a machine-learning model, such as an artificial neural network (see pp.76), can be evaluated by its "cost function". This is a measure of the change that occurs during training between the actual outputs from the model and the outputs expected by the programmer. This difference, called the "cost", is expressed as a number. The higher the number, the greater the gap between the real and the anticipated outputs, and the poorer the model. As the model learns, the cost reduces and performance improves. The training is complete when the cost is zero, or as close to zero as possible.

IMPROVING PERFORMANCE

A machine-learning model improves its performance by fine-tuning its settings. Instead of having to process huge amounts of data, the model can start at a random data point and then "nudge" its way towards a better solution. The algorithm that trains it to do this is known as the "gradient descent". Each time the model adjusts its settings, the gradient descent rates its success using the "cost function" (see opposite). Plotting the gradient of the cost function on a graph reveals a curve. The model reduces its cost function by following the steepest downward slope. When the slope levels off, the model is as good as it can be and it stops learning.



1. Test the model
Run the ANN using training data (see p.61). The output is called the "test data".

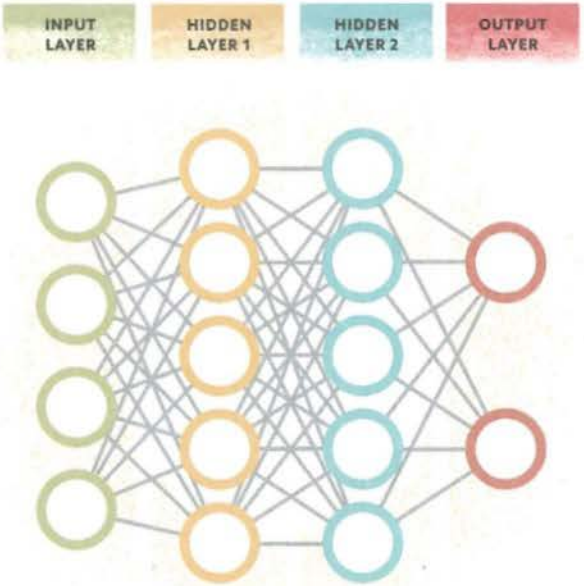
2. Calculate the cost function
Compare the test data to the training data. The difference is the cost function (see p.80).

REFINING THE MODEL

The "delta rule" – also known as the delta learning rule – enables a single-layer artificial neural network (ANN, see pp.76) to boost its performance by refining its settings. It makes use of "gradient descent" (see p.81) to identify the best choices for improving the model. As the model's output gets nearer to the expected output, smaller adjustments are carried out, until the outputs are as close as possible to each other. Backpropagation (see p.84) is a generalized form of the delta rule that applies to ANNs with any number of layers.

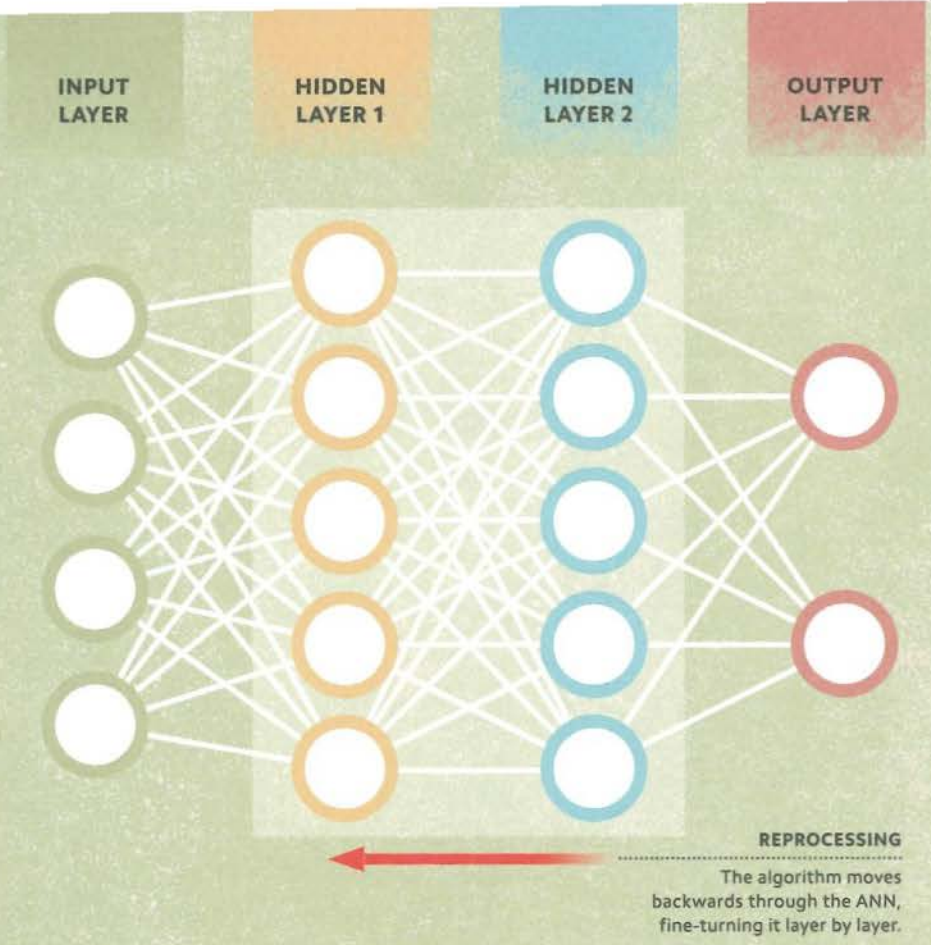
3. Use a gradient descent algorithm
This determines which direction the model's settings should move for a lower cost function.

4. Update the model
Amend the settings in the ANN based on the feedback from the gradient descent.



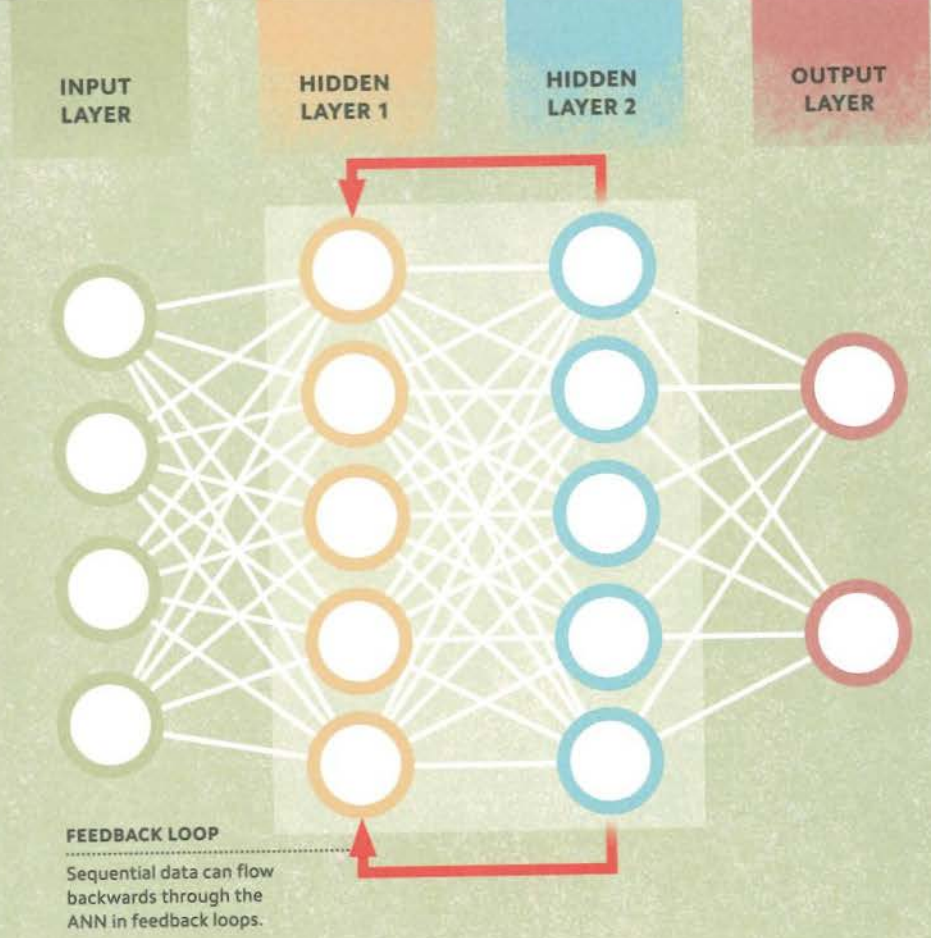
A ONE-WAY NETWORK

A "feedforward neural network" (FNN) is a simple artificial neural network (ANN, see p.76) in which information flows forwards only – from the input layer, through the hidden layers, to the output layer. The connections between the nodes in an FNN do not form "feedback loops" – in other words, outputs are not fed backwards as inputs, as they are in a recurrent neural network (RNN, see p.85). The most basic form of feedforward neural network is a single artificial neuron (see p.21), which can undergo machine learning using gradient descent (see p.81).



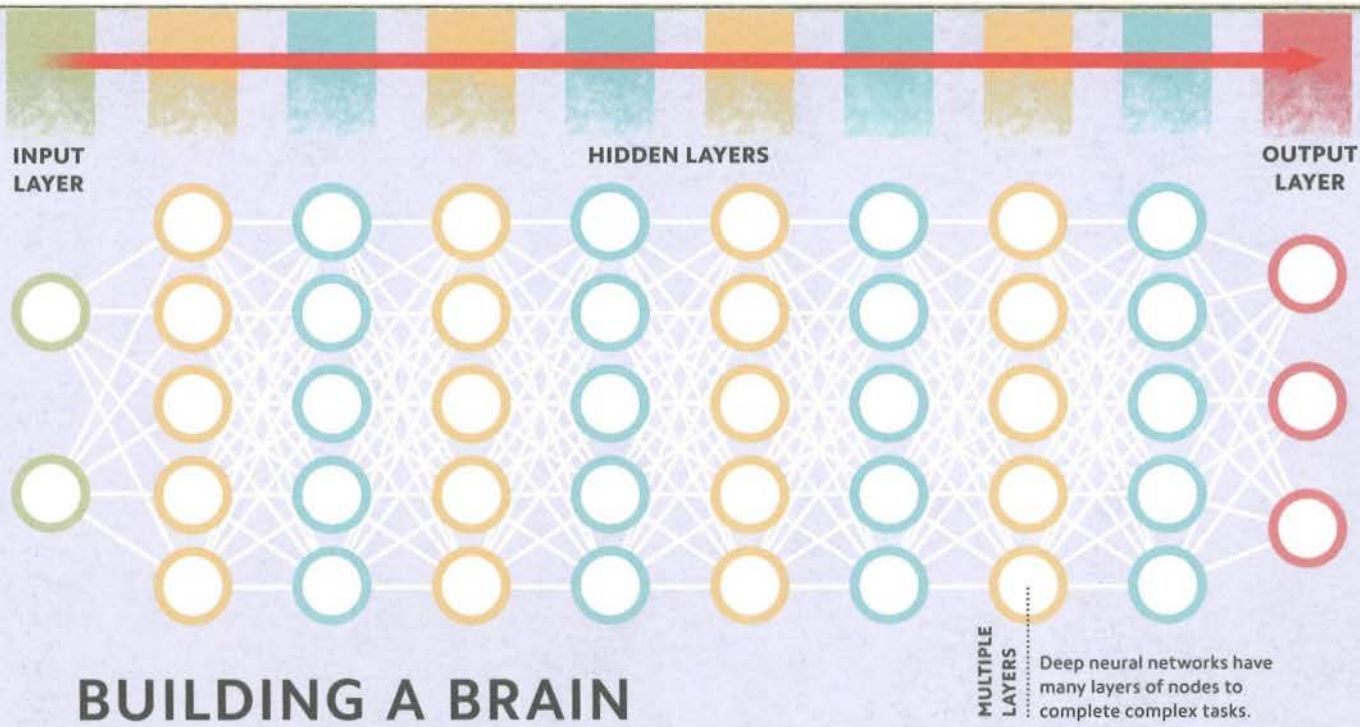
FINE-TUNING DATA

Backpropagation is a type of algorithm that is used to train artificial neural networks (ANNs, see p.76), specifically feedforward neural networks (see p.83). It is known as backpropagation because it begins at the final (output) layer and moves in reverse towards the first (input) layer. During this process, nodes are reprogrammed by adjusting their weights (see p.78) and biases (see p.79), using gradient descent (see p.81) to find out whether increasing or decreasing them will produce better results. This has the effect of fine-tuning the ANN to produce more accurate outcomes overall.



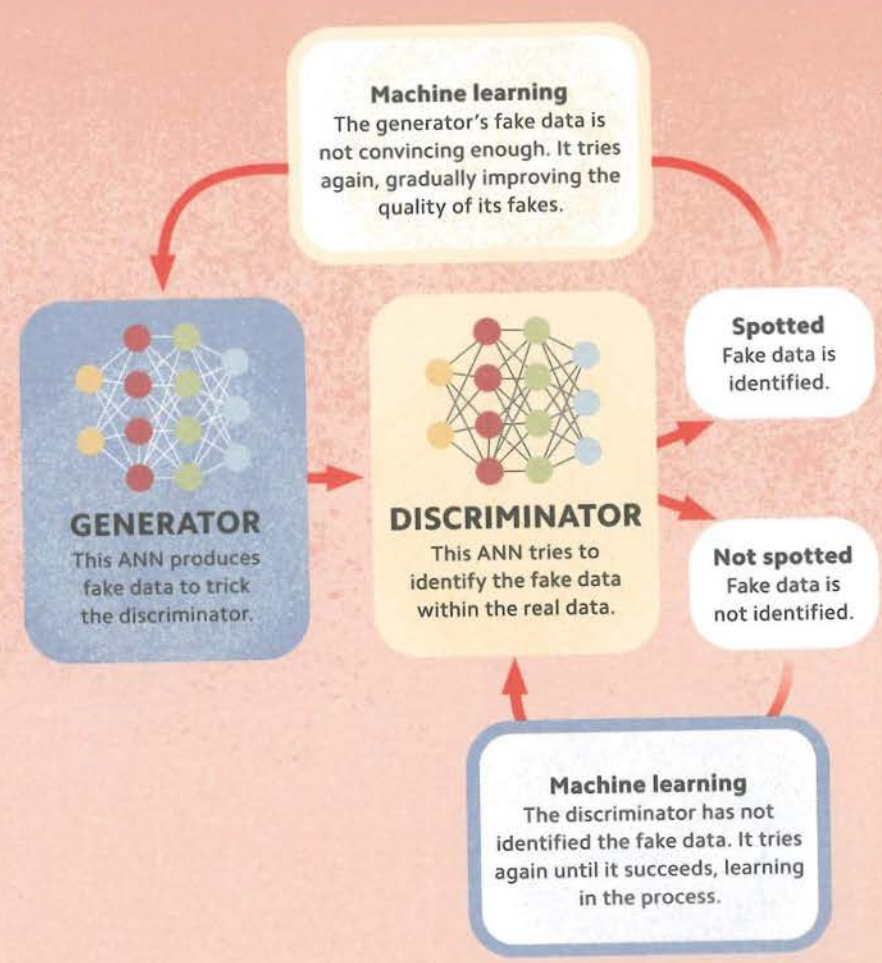
STRUCTURED DATA

A recurrent neural network (RNN) is a type of ANN in which data can move backwards in a "feedback loop". RNNs are used to process sequential data – data that has to be in a specific order – such as language. While traditional ANNs process individual data points to give an outcome, RNNs maintain the essential structure and relationships within sequential data, so that it remains intact. In doing so, RNNs can be used to predict the next output of a sequence. They are used widely in natural language processing tasks (see p.112), including training virtual assistants to carry out spoken conversations.



BUILDING A BRAIN

Deep learning is a powerful form of machine learning based on artificial neural networks (ANNs, see p.76). It uses ANNs with many hidden layers, known as deep neural networks (DNNs), to identify increasingly more meaningful features from input data. As with ANNs, data passes from the input layer into the hidden layers, where the nodes receive, process, and pass it on to another node in the next layer. With so many layers, DNNs process data very accurately and quickly, and are also capable of making accurate predictions. They are used in many complex AI processes, such as natural language processing (see p.112).

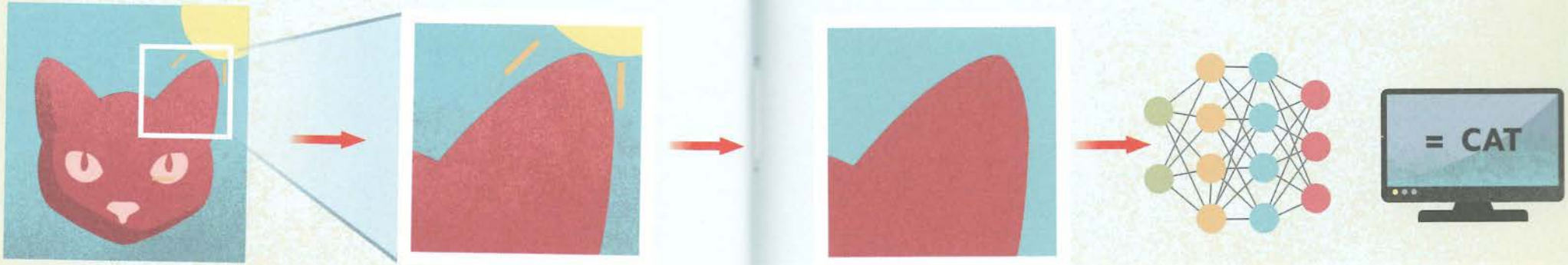


AI VS AI

A generative adversarial network (GAN) is a machine-learning model that uses two competing artificial neural networks (ANNs, see p.76). One ANN, the "generator", uses unlabelled training data (see p.61), supplied by the programmer, to create new fake data that it supplies to the second ANN – the "discriminator". The discriminator aims to identify the fake data. If it succeeds, the generator tries again, creating fake data that is harder to distinguish from real data. If the discriminator fails, it tries again to identify fake data more effectively. This process continues until the generator can make convincing fake data. In other words, it can use existing data to create accurate new data, such as predictions.

PROCESSING VISUAL DATA

A convolutional neural network (CNN) is a type of deep neural network (see p.86) that is similar to the structure of the visual cortex: the part of the brain that takes and analyses information from the eye. CNNs are effective tools for computer vision (see p.110), since they can be taught to recognize features in input images, such as the pointed ears of cats. There are three types of layers (see p.77) in a CNN. The first type performs a function called a "convolution", which allows features in an image to be detected. These layers first extract low-level features (lines and edges), before extracting higher-level features (shapes). They work by passing a filter over the image that creates a "map" of the location of each feature on the image. Between each convolution, there is a "pooling" layer, which reduces the complexity of the feature maps. The data from these layers is flattened, and then passes through a "classification" layer (see p.66), which identifies and labels the image.



Input
The input in CNNs is typically an image – such as a photograph of a cat.

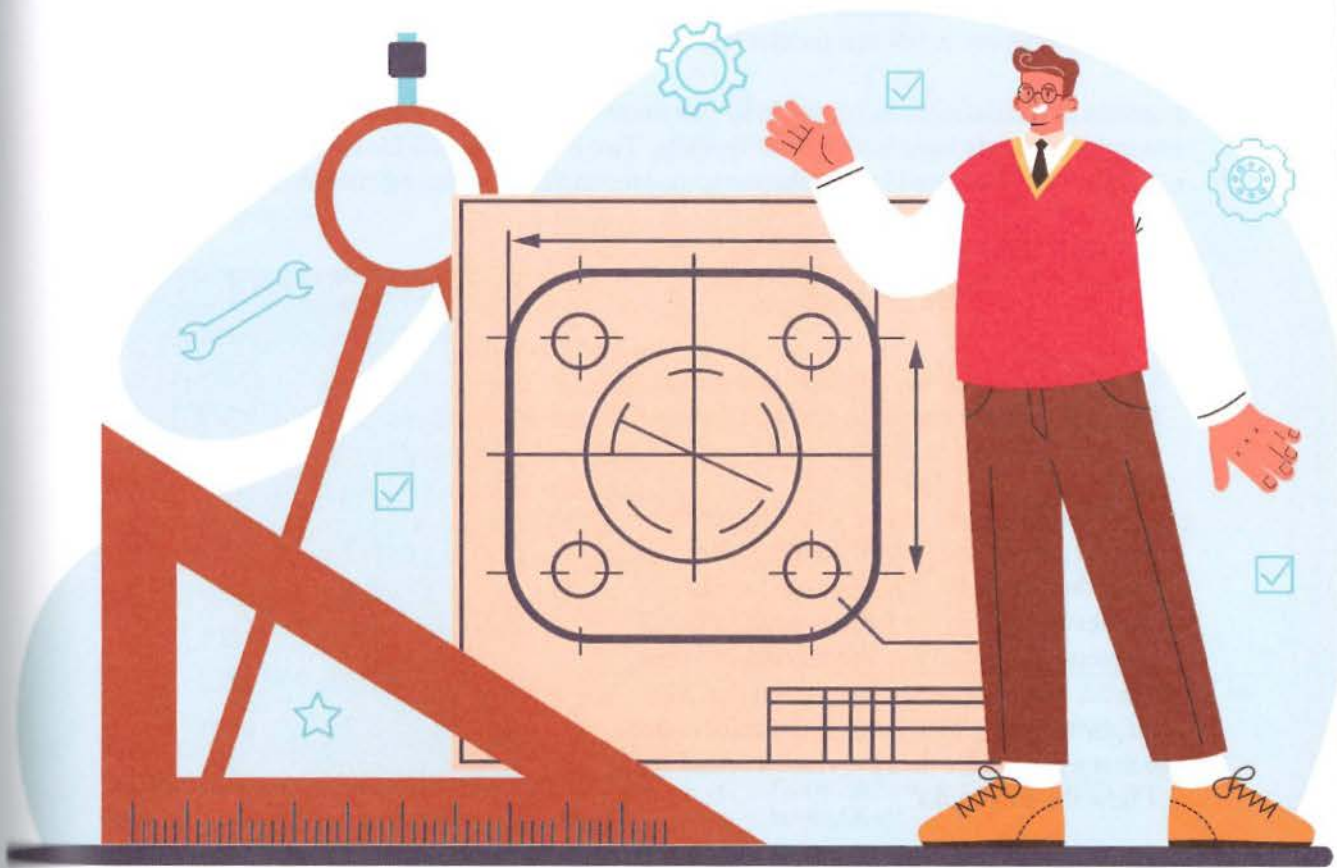
Convolution
A filter is applied to the image to produce feature maps. This enables features to be detected in patterns of pixels.

Pooling
"Mess" is cut out to reduce the amount of computing power required, and the features are abstracted.

Classification
Through a process of classification, the AI associates the data from the previous layers with an image.

Output
The AI identifies the photograph as being one of a cat.

> **"I get very excited when we discover a way of making neural networks better – and when that's closely related to how the brain works."**
> Geoffrey Hinton



CHAPTER

7

K Nearest Neighbors

In this chapter, a very popular algorithm called KNN is explored. KNN is short for K nearest neighbors. This algorithm is quite popular and very easy to understand.

Datasets for K Nearest Neighbor

KNN algorithm can be used for both classification and regression types of problems. KNN is used in supervised learning which means the training data for the algorithm contains both features and labels. KNN can predict both, numbers and categories.

Features of the dataset can be anything like numbers, categories, text etc. In the real world examples, datasets have hundreds of features. Two examples, one for classification and one for regression are presented in this chapter to understand how K nearest neighbors works.

How does a KNN predict

KNN can predict both, numbers and categories. The first example shown in Figure 7.1 is one where KNN predicts categories.

In this data set there are only two features, *Years in school* and *height*, and the label is whether someone is an *adult* or a *child*. Whenever a row in the table is adult, it is represented by a red dot and whenever a row is child, it is represented by a green dot. The categories in the label column are color coded so that a plot of the data provides a visual sense of how the data looks.

Years in school	height	label
0	2.18	child
7	5.58	adult
4	5.95	adult
4	4.36	child
2	5.53	adult
9	5.38	child
11	5.29	adult
4	5.91	adult
0	3.96	child
1	2.72	child

Figure 7.1 Example dataset with two features and a label

Plot of this dataset is shown in Figure 7.2. Column *Years in school* is on the x-axis, and *height* is on the y-axis.
All the red points in the plot are adults, and all the green points are children. A visual inspection of the pattern of points in the plot shows that if the height is a small number, then most likely it's a child. If *Years in school* is a large number, then its most likely an adult, even if the height is a small number. This is the kind of visual insight that plotting the data can provide.

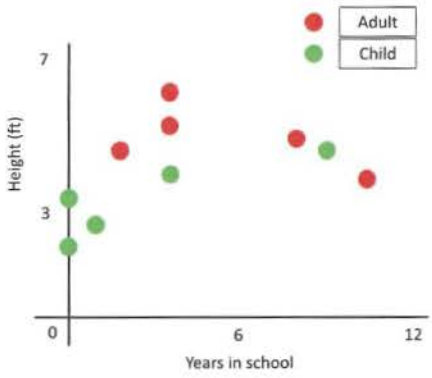


Figure 7.2 Plot of the dataset with features *Years in school* on the x-axis and label *height* on the y-axis

The way a K nearest neighbors algorithm make predictions is easy to understand using the plot. KNN looks at the training data provided and memorizes it. It uses this information to make future predictions. For example, if the KNN algorithm has to make a prediction on whether someone is an adult or a child, like the new point shown in Figure 7.3, it looks at this point in the context of the training data around it.

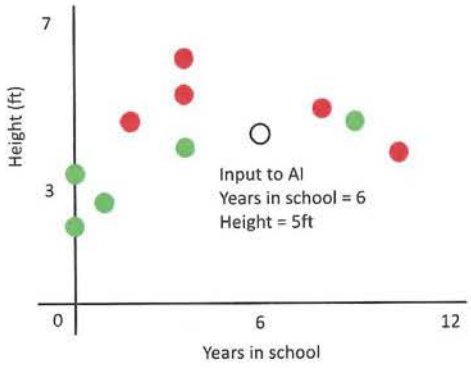


Figure 7.3 A new point shown by the uncolored circle in the plot too illustrate how KNN makes predictions

The new point shown in the figure has *Years in school* around six and height around five. The reason this point is not colored is that the new point is an adult or a child. A KNN algorithm that has memorized the training data will look for K nearest neighbors to the point. If K is assume to be one, KNN will look at one nearest point as shown in Figure 7.4.

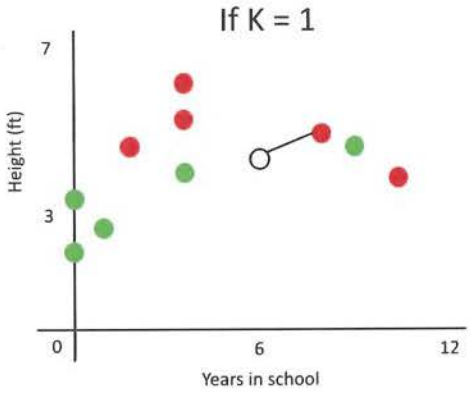


Figure 7.4 Predictions using a KNN model for a value of K=1.

Since a red point, which represents an adult is the closest neighbor, KNN will predict the new point to be an adult. This is how KNN makes predictions. However, the value of K here can be chosen by the user.

If K is three instead of one, KNN algorithm looks at three neighbors. Figure 7.5 shows the three closest neighbors. In this case, note that two neighbors are green and one neighbor is red. The algorithm picks green, which is a child as its prediction. This shows that in a K nearest neighbors algorithm, depending on the value of K chosen by the user, the algorithm may predict an adult or a child.

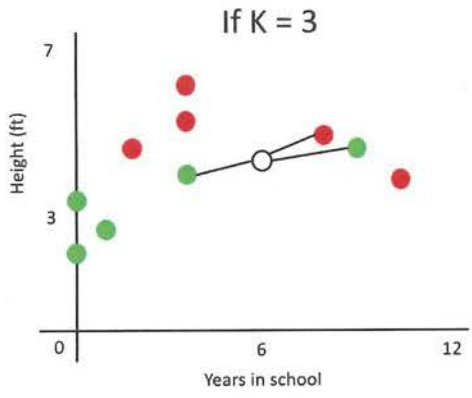


Figure 7.5 Predictions using a KNN model for a value of K=3.

Figure 7.6 shows an example of a regression data set. A data set is regression, when the prediction variable is a continuous number. For example, in this case, price of a house is a continuous number and not a category. Price of a house could be any number and hence it's a regression problem.

Plotting a regression dataset is a little bit tricky. It's different from classification, where each category can be color coded. In regression, there are no categories. Using different colors for each unique value will be too many colors and can be visually confusing. A good way to plot for a case like this is to assign every point a number. There is no color coding, but every point has a unique number assigned in the column *index* in Figure 7.6. This table is much bigger than the previous one because there is a new column called an index, which is here simply to identify the sample. It's not a feature, the features are square footage of the house represented by the column *sq_ft* and the *age* of the house. The AI should predict the label which is the *price* of the house.

There are 10 points, and every point represents a single sample plotted in Figure 7.7. KNN follows exactly the same principle that it followed in classification. If a new house comes in, which is around 600 square feet and 28 years old, how does a KNN algorithm predict this value?. If the value of K is three, it is going to look at who the nearest neighbors are. Here, it turns out that the samples with index seven, eight, and nine are the nearest neighbors.

In this case, what should the price of this house be?

index	sq_ft	age	price
1	**895**	26	707158
2	**827**	61	285327
3	**873**	58	365744
4	**847**	58	338163
5	**801**	66	204300
6	**522**	58	14117
7	**784**	46	388363
8	**450**	25	121220
9	**657**	33	381713
10	**425**	52	82357

Figure 7.6 Example regression dataset with two features sq_ft, age and a label price.

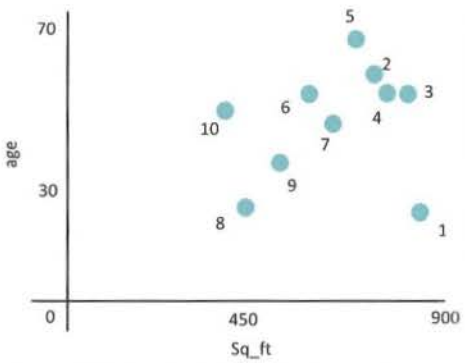


Figure 7.7 Plot of the dataset in Figure 7.6

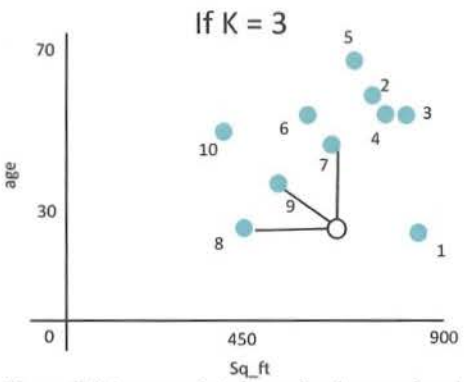


Figure 7.8 A new point shown by the uncolored circle in the plot to illustrate how KNN makes predictions

In the real world, it turns out that there are several different ways to do this. A really simple way is to take the average. Average of the seventh house, the eighth house and the ninth house would be the price of the new house. These are the ways that an AI predicts in the case of KNN for classification, and regression.

Hyper-parameters of KNN

K is a hyper-parameter of KNN. The reason it is called a hyper-parameter is because this value is set by the user and has an impact on how the algorithm behaves. An important question is how do you find the best K?. The process of finding the best K is called hyper parameter tuning. It's very hard to give a single value of K as an answer. Every dataset is unique and part of the puzzle is to find out by simply trying out different values. It's something to experiment and find out. Whichever value gives the best performance is the best K for the dataset. The process of figuring out this value of K is called hyper parameter tuning.
In summary, KNN is a very simple algorithm and is very easy to understand. It has a single hype-parameter called K that can be tuned. There are many algorithms that are more sophisticated and have many hyper parameters to tune. In such a case, hyper-parameter tuning can be very complex. KNN has the advantage of being really simple, easy to understand, and easy to hyper parameter tune.

Pitfalls of KNN

Exercise: Guess the House Price

House one is 1000 square feet and zero years old, house two is 800 square feet eight and zero years old. House three is 1000 square feet and 100 years old. The label value is the price of the house. House number two is $10,000 and house number three is $5,000. As a human, if you were to predict the price of house one and you have only two options to pick from, either $10,000 or $5,000, what would you pick?

Square feet	age	Price
1000	0	
800	0	10,000$
1000	100	5,000$

Figure 7.9 A table with two features Square feet and age, one label Price and just 2 samples in training data.

Let's look at what KNN would pick. If K is equal to one, what is the closest neighbor in terms of the features, which is square footage and age here. It turns out that the distance between house one and house two is 200. The distance between house one and house three is only 100. So, it would pick house 3, whose price is 5000$.

Table in Figure 7.9 is an illustration of one of the pitfalls of KNN. A house that is 100 years old is actually pretty cheap compared to a house which is brand new. The age of a house is much more important than the square footage.

However, KNN completely misses this. The reason it misses it is it does not know that square footage is more important or less important than age, it rates all features equally. However, square footage can be anywhere from like 800 to 3000, but age can only be between 0 to 200. People usually do not live in a 1000 year old house. Typically, houses are between 0 and 200. Square footage on the other hand, can be from 800 to 3000. Clearly, the scale is different. The numbers start differently. Square footage does not start at zero, but age starts at zero. So, the starting point of these features is also different. In addition, the range is also different. As a result, KNN can sometimes give really bad results because it does not know which feature is more important. This is a drawback of KNN to keep in mind. If an awesome data set that is supposed to do really well with the AI, is giving bad predictions, then think about whether the features in the dataset have different scales.

There are a lot of things that researchers have done to make sure that one can indirectly tell KNN that one feature is more important than the other. For the specific example in Figure 7.9, one really easy way to do this is, multiply all values of age with 1000. Then the value of age is between zero and 200,000 instead of being between 0 and 200. Such multiplication with a constant number artificially increases the range of age. Alternatively, one can divide all the values of square feet by a constant number to make sure that they are more or less equal in range. This is one example of how the drawback of KNN can be overcome.
Another important pitfall of KNN to keep in mind is that it can only predict a value that is in the range of training data. If for example, the KNN needs to predict the price of a house whose square footage is much larger than the maximum value present in training data, KNN can only refer to the closest examples in training data. It will not be able to scale up the price of the house proportionally.

Assessment

1. What types of problems can KNN make predictions for? (a) Regression only (b) Classification only (c) Classification and Regression (d) None

2. What does the K in the KNN algorithm represent ?

3. How would a KNN algorithm behave when the value of K is equal to the number of samples in the training dataset?

4. If a training dataset has three features, *volume*, *weight* and *viscosity* and the label *density*, what can you do to make the algorithm give more importance to the feature *volume* than others, when it makes predictions?.

5. KNN is a type of unsupervised algorithm (a) True (b) False.

6. Fill in the blanks

(a) KNN algorithm predictions may change when you vary the value of _____.

(b) KNN is short for _____.

(c) Hyper-parameter of the KNN algorithm is called _____ .

7. A KNN algorithm can only have two features in the dataset (a) True (b) False

8. A training dataset for regression has labels in the range 100 and 200. Can a KNN algorithm predict a label value of 300 for any combination of feature values?

9. I have a dataset, where the label is *yes* or *no* based on three features, all of which are numerical. Can a KNN model be used here to make predictions? Please explain.

10. What are some drawbacks of a KNN algorithm?